

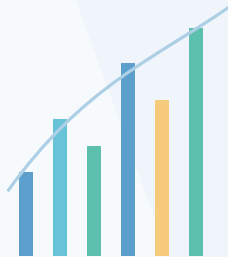


BIG DATA ANALYTICS IN FINANCE

Lecture 2: Data Transform

Scaling, encoding, distribution transforms, feature expansion, and target transformation

Dr. Nguyen Tuan Long National Economics University | 2026



Why Do We Need to Transform Data?

Model expectations

- ▶ Most algorithms do not work well with unprocessed raw data.
- ▶ Models require numerical inputs rather than text labels.
- ▶ SVM, KNN, and linear models are sensitive to feature scale.
- ▶ Some models benefit when distributions are closer to Gaussian.

Goal

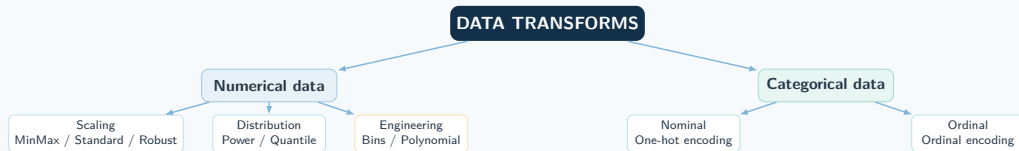
Convert raw data into a well-structured representation that improves learning, evaluation, and deployment.

Scale mismatch example

Feature	Value
Annual income	50,000
Experience	5

Without scaling, distance calculations can be dominated by income.

Overview of Data Transform



A transformation is learned from data

Parameters such as minima, means, quantiles, category sets, and power coefficients must be fitted on training data only.

Agenda

Part I: Feature Transformation

1. Scaling numerical data
2. Encoding categorical data
3. Labs with Diabetes and Breast Cancer data

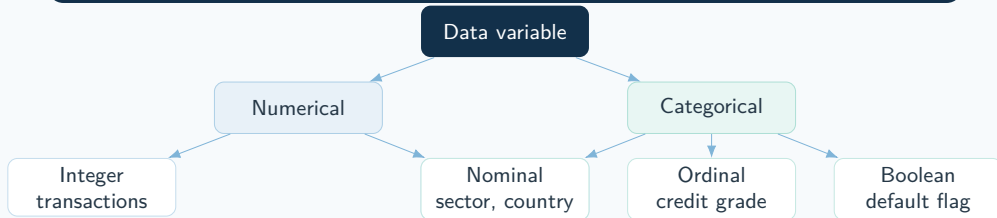
Part II: Distribution and Other Transforms

4. Power and quantile transforms
5. Discretization and polynomial features
6. Transforming the target variable

Teaching sequence

Concept → toy calculation → Scikit-learn tool → dataset practice → leakage-safe workflow.

PART I Overview of Data Variable Types



Part I: Scaling Numerical Data

SCALING Bring Features to a Comparable Scale

Problem

Features measured in different units can distort distances, gradients, and regularization penalties.

Key techniques

- ▶ MinMaxScaler: fixed range
- ▶ StandardScaler: mean 0, standard deviation 1
- ▶ RobustScaler: median and IQR

Important distinction

Scaling changes location and magnitude, but it does not automatically remove skewness or make a distribution Gaussian.

Scaling Technique 1: MinMaxScaler

Definition and formula

Maps values to a specified range, usually $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

When to use

- ▶ Distance-based methods such as KNN
- ▶ Bounded inputs such as pixel intensity
- ▶ Situations where a fixed output range is meaningful

Major drawback

Min and max are highly sensitive to outliers. One extreme observation can compress most values into a narrow interval.

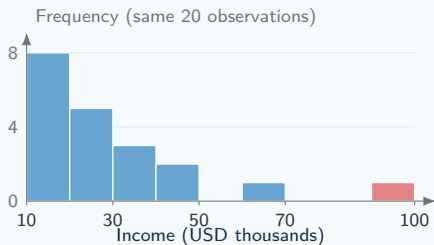
Test values outside training range

New values may transform below 0 or above 1. Use `clip=True` only when clipping is substantively justified.

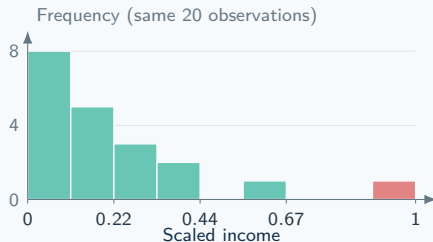
MinMaxScaler: New Range, Same Distribution Shape

Same right-skewed training sample throughout: most values are small; one value is 100.

Before: original income



After: MinMaxScaler



$$x' = \frac{x-10}{100-10} \quad 10 \mapsto 0 \quad 20 \mapsto 0.111 \quad 100 \mapsto 1$$

Read the shape, not just the axis labels

The right tail remains: scaling to $[0, 1]$ does **not** make the data uniform. The extreme value still compresses ordinary values into the lower part of the range.

Matching bin edges are transformed with the data; bar counts stay identical. Default scaling, without clipping.

MinMaxScaler: Toy Dataset and Hand Calculation

Toy income data

For training values (20, 30, 50):

$$x_{\min} = 20, \quad x_{\max} = 50.$$

$$20 \mapsto 0, \quad 30 \mapsto \frac{1}{3}, \quad 50 \mapsto 1.$$

A new value 60 maps to $40/30 = 1.33$.

Result table

Income	20	30	50	60 (new)
Scaled	0	.333	1	1.333

Question

What happens when test data fall outside the training minimum–maximum range?

MinMaxScaler: Tool and Implementation

Tool

`sklearn.preprocessing.
MinMaxScaler`

Workflow

1. Fit on training data.
2. Transform training data.
3. Reuse the fitted scaler for test data.

```
from sklearn.preprocessing import MinMaxScaler

train = np.array([20, 30, 50]).reshape(-1, 1)
test = np.array([60]).reshape(-1, 1)

scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(train)
X_test_scaled = scaler.transform(test)

print(scaler.data_min_, scaler.data_max_)
print(X_train_scaled.ravel())
print(X_test_scaled.ravel())
```

Scaling Technique 2: StandardScaler

Definition and formula

Centers a feature at zero and scales it to unit standard deviation:

$$z = \frac{x - \mu}{\sigma}.$$

When to use

- ▶ A strong default for many numerical workflows
- ▶ Linear and logistic regression
- ▶ SVM, PCA, and regularized models

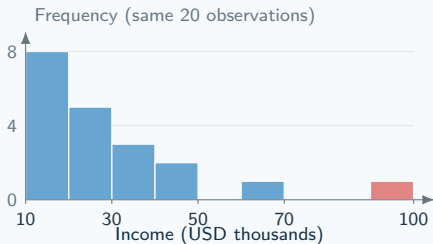
What it does not do

Standardization does not change the distribution's shape. A skewed feature remains skewed after centering and scaling.

StandardScaler: Mean Zero Does Not Mean Gaussian

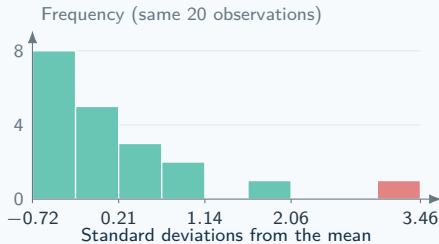
Same 20 observations: $\mu = 25.5$, $\sigma \approx 21.56$ (population standard deviation).

Before: original income



$$z \approx \frac{x-25.5}{21.56} \quad \mu \mapsto 0 \quad \mu + \sigma \mapsto 1 \quad 100 \mapsto 3.46$$

After: StandardScaler



Centered and rescaled, but still right-skewed

The training mean becomes 0 and standard deviation becomes 1. The asymmetric shape and extreme observation remain; no bell curve is created.

Matching bin edges are transformed with the data; bar counts stay identical. Mean and standard deviation are still sensitive to extremes.

StandardScaler: Toy Dataset and Hand Calculation

Toy data (2, 4, 6)

$$\mu = 4, \quad \sigma = \sqrt{8/3} \approx 1.633.$$

$$z \approx (-1.225, 0, 1.225).$$

The ordering and distribution shape are unchanged.

Question

Does StandardScaler make a skewed distribution normal? **No.**

Interpret the standardized values

- ▶ $z = 0$: equal to the training mean
- ▶ $z = 1$: one training standard deviation above the mean
- ▶ $z = -1$: one standard deviation below the mean

Test value example

A new value 8 maps to $(8 - 4)/1.633 \approx 2.45$ using training statistics.

StandardScaler: Tool and Implementation

Tool

`sklearn.preprocessing.
StandardScaler`

Inspect fitted parameters

`mean_`, `var_`, and `scale_` are learned only from training observations.

```
from sklearn.preprocessing import StandardScaler

train = np.array([2, 4, 6]).reshape(-1, 1)
test = np.array([8]).reshape(-1, 1)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(train)
X_test_scaled = scaler.transform(test)

print(scaler.mean_, scaler.scale_)
print(X_train_scaled.ravel())
print(X_test_scaled.ravel())
```

Scaling Technique 3: RobustScaler

Definition and formula

Uses robust statistics rather than mean and standard deviation:

$$x' = \frac{x - Q_2}{Q_3 - Q_1} = \frac{x - \text{median}(X)}{IQR}.$$

Why it is robust

Median and quartiles change much less than the mean and standard deviation when extreme observations are present.

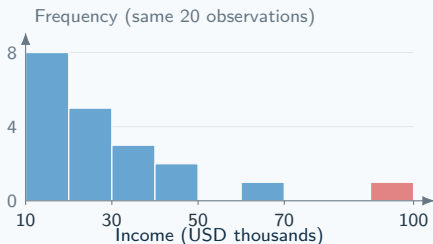
When to use

Prefer RobustScaler when genuine extreme values are common and should not determine the scale of ordinary observations.

RobustScaler: Robust Scale, Not Outlier Removal

Same 20 observations: $Q_1 = 10$, median = 20, $Q_3 = 30$; hence $IQR = 20$.

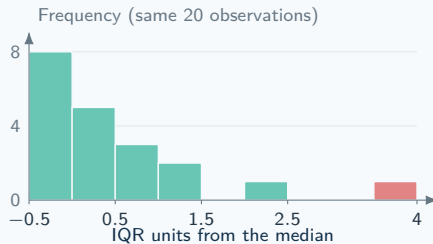
Before: original income



$$x' = \frac{x-20}{20}$$

$$Q_1 \mapsto -0.5 \quad Q_2 \mapsto 0 \quad Q_3 \mapsto 0.5 \quad 100 \mapsto 4$$

After: RobustScaler



The red bar is still there

The training median becomes 0 and IQR becomes 1. “Robust” describes the fitted center and scale, not removal of the tail or extreme values.

All three are positive affine transforms: location and spread change, not shape. Histogram frequency is shown, not probability density.

RobustScaler: Toy Dataset and Hand Calculation

Data with an outlier: (10, 12, 14, 16, 100)

Method	10	12	14	16	100
MinMax	0	.022	.044	.067	1
Robust	-1	-.5	0	.5	21.5

MinMax compresses ordinary values; RobustScaler preserves their relative spacing.

Calculate robust statistics

$$Q_1 = 12, \quad Q_2 = 14, \quad Q_3 = 16,$$

$$IQR = 4, \quad 16 \mapsto \frac{16 - 14}{4} = 0.5.$$

Question

How do all three scalers behave on the same data when an outlier is present?

RobustScaler: Tool and Implementation

Tool

`sklearn.preprocessing.
RobustScaler`

Fitted parameters

The transformer stores the training median in `center_` and the selected quantile range in `scale_`.

```
from sklearn.preprocessing import RobustScaler

train = np.array([10, 12, 14, 16, 100]).reshape(-1, 1)

scaler = RobustScaler()
X_scaled = scaler.fit_transform(train)

print('Median:', scaler.center_)
print('IQR:', scaler.scale_)
print(X_scaled.ravel())
```

Scaling: Which One Should You Choose?

Scaler	Best fit	Main caution
MinMaxScaler	Fixed range, KNN, bounded inputs	Highly sensitive to extremes
StandardScaler	Strong default; linear, SVM, PCA	Mean and SD are outlier-sensitive
RobustScaler	Features with meaningful outliers	Output is not bounded

Golden rule: prevent data leakage

Split first. Fit the scaler on training data only, then use the fitted scaler to transform validation, test, and production data.

Full-Data Practice: Diabetes Dataset

Dataset

The Scikit-learn Diabetes dataset contains 10 physiological features from 442 patients. The target measures disease progression one year later.

Lab task

Compare the three scalers using summary statistics, plots, and a downstream regression model.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_diabetes

diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target
feature_names = diabetes.feature_names
df = pd.DataFrame(X, columns=feature_names)
```

Encoding Categorical Data

Ordinal categories

Categories have a meaningful order:

Small < Medium < Large.

Use an encoding that preserves the known order.

Nominal categories

Categories have no intrinsic ranking, for example country, sector, or color. Use separate indicators rather than arbitrary integers.

Why encoding matters

Machine-learning estimators generally require numerical arrays, but the numerical representation must preserve the meaning of the original categories.

Encoding: Toy Customer Dataset

Raw observations

Customer	Risk grade	Region
C1	Low	North
C2	Medium	South
C3	High	North
C4	Medium	Central

Identify the variable type

- ▶ Risk grade is **ordinal**: Low < Medium < High.
- ▶ Region is **nominal**: no region is numerically greater than another.

Required transforms

Ordinal-encode risk grade and one-hot encode region.

Encoding: Hand Calculation

Ordinal mapping

Define the business order:

Low $\mapsto$ 0, Medium $\mapsto$ 1, High $\mapsto$ 2.

Thus C1–C4 become (0, 1, 2, 1).

One-hot mapping

Region	Central	North	South
North	0	1	0
South	0	0	1
North	0	1	0
Central	1	0	0

Interpretation

The ordinal column preserves order; the three region indicators avoid inventing an order.

OrdinalEncoder: Tool and Implementation

What it does

Maps ordered categories to integers, for example $S \mapsto 0$, $M \mapsto 1$, $L \mapsto 2$, $XL \mapsto 3$.

Warning

Applying ordinal encoding to nominal data introduces a false ranking and artificial distances.

```
from sklearn.preprocessing import OrdinalEncoder

df_sizes = pd.DataFrame({
    'size': ['S', 'M', 'L', 'XL', 'M', 'S']
})
size_order = ['S', 'M', 'L', 'XL']
encoder = OrdinalEncoder(categories=[size_order])
encoded = encoder.fit_transform(df_sizes[['size']])
```

Question

If order is not specified, Scikit-learn infers categories and usually sorts them lexicographically—not substantively.

OneHotEncoder: Tool and Implementation

What it does

Creates one binary column per nominal category:

Red $\mapsto (1, 0, 0)$, Green $\mapsto (0, 1, 0)$.

High cardinality

A feature with many categories can create a very wide sparse matrix and increase overfitting risk.

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(
    sparse_output=False,
    handle_unknown='ignore'
)
encoded = encoder.fit_transform(
    df_colors[['color']]
)
```

Pandas or Scikit-learn?

`pd.get_dummies` is convenient for exploration; `OneHotEncoder` integrates safely with pipelines and unseen categories.

Full-Data Practice: Breast Cancer Dataset

Dataset

The UCI Breast Cancer dataset records categorical tumor and patient characteristics. The task is to classify recurrence events.

```
url = ('https://archive.ics.uci.edu/ml/machine-learning-databases/'  
      'breast-cancer/breast-cancer.data')  
columns = [  
    'Class', 'age', 'menopause', 'tumor-size', 'inv-nodes',  
    'node-caps', 'deg-malig', 'breast', 'breast-quad', 'irradiat'  
]  
df_cancer = pd.read_csv(  
    url, header=None, names=columns, na_values='?'  
)
```

Lab task

Identify ordinal and nominal columns, impute missing categories, encode them appropriately, and place all steps inside a ColumnTransformer pipeline.

Part II: Distribution Transforms

PART II Change the Shape of a Feature Distribution

Why transform distributions?

- ▶ Reduce strong skewness
- ▶ Stabilize variance
- ▶ Reduce the influence of long tails
- ▶ Better align with modeling assumptions

Key techniques

- ▶ PowerTransformer: Box–Cox and Yeo–Johnson
- ▶ QuantileTransformer: uniform or normal output

Not the same as scaling

Distribution transforms alter shape; scalars primarily alter location and magnitude.

Distribution Transform 1: PowerTransformer

Box–Cox: positive data only

$$y^{(\lambda)} = \begin{cases} \frac{y^\lambda - 1}{\lambda}, & \lambda \neq 0, \\ \log(y), & \lambda = 0. \end{cases}$$

Restriction

Every value must be strictly positive: $y > 0$.

Yeo–Johnson: flexible default

Extends the power-transform idea to positive, zero, and negative values. The coefficient λ is estimated from training data, commonly by maximum likelihood.

Purpose

Find a monotonic transformation that makes the feature less skewed and closer to Gaussian.

Power Transform: Toy Claim Dataset

Right-skewed claim amounts

Consider positive claims in USD thousands:

$$y = (1, 4, 9, 25, 100).$$

The final claim is much farther from 25 than the early observations are from one another.

Transformation objective

Compress the long right tail while preserving order. Box-Cox can be used because all values are strictly positive.

Toy coefficient

Use $\lambda = 0.5$ for a transparent hand calculation. In practice, the tool estimates λ from training data.

Power Transform: Hand Calculation

Box-Cox with $\lambda = 0.5$

$$y^{(0.5)} = \frac{y^{0.5} - 1}{0.5} = 2(\sqrt{y} - 1).$$
$$(1, 4, 9, 25, 100) \mapsto (0, 2, 4, 8, 18).$$

Effect

The original gap $100 - 25 = 75$ becomes $18 - 8 = 10$. Large values are compressed more strongly than small values.

If values include zero or negatives
Use Yeo-Johnson rather than Box-Cox.

PowerTransformer: Tool and Implementation

```
from sklearn.preprocessing import PowerTransformer

pt_boxcox = PowerTransformer(
    method='box-cox', standardize=False
)
boxcox_data = pt_boxcox.fit_transform(
    skewed_data.reshape(-1, 1)
)

pt_yj = PowerTransformer(
    method='yeo-johnson', standardize=True
)
yj_data = pt_yj.fit_transform(
    data_with_neg.reshape(-1, 1)
)
```

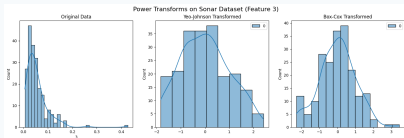
Box-Cox with zero or negative data

Do not silently apply Box-Cox. Either use Yeo-Johnson or justify a positive shift based on the domain.

What does `standardize=True` do?

After the power transform, Scikit-learn also centers and scales the output to zero mean and unit variance.

Full-Data Practice: Power Transform on Sonar



```
url_sonar = (  
    'https://archive.ics.uci.edu/ml/machine-learning-databases/'  
    'undocumented/connectionist-bench/sonar/sonar.all-data'  
)  
df_sonar = pd.read_csv(url_sonar, header=None)  
X_sonar = df_sonar.iloc[:, :-1]
```

Dataset

Sixty continuous sonar-energy features are used to distinguish metal objects from rocks underwater.

Lab task

Compare original, Box-Cox, and Yeo-Johnson distributions; record estimated λ values and downstream model performance.

Distribution Transform 2: QuantileTransformer

Two output distributions

- ▶ **Uniform:** maps empirical ranks into $[0, 1]$.
- ▶ **Normal:** maps ranks through the Gaussian quantile function.

When to use

Complex, multimodal, or outlier-heavy distributions that power transforms cannot adequately regularize.

Why it helps

Rank-based mapping prevents observations from clustering too tightly and can help distance-based models.

Warning

The transformation is strongly nonlinear and can obscure meaningful spacing in the original units.

Quantile Transform: Toy Transaction Dataset

Observed transaction values

$$x = (10, 20, 30, 100).$$

The values have equal ranks 1–4, although the final numerical gap is much larger.

Rank-based idea

QuantileTransformer replaces raw spacing with positions in the empirical cumulative distribution.

Consequence

Order is preserved, but the magnitude of the gap between 30 and 100 is intentionally discarded.

Quantile Transform: Hand Calculation

Uniform output

With four sorted observations, assign positions

$$u_i = \frac{\text{rank}(x_i) - 1}{n - 1}.$$

Therefore

$$(10, 20, 30, 100) \mapsto \left(0, \frac{1}{3}, \frac{2}{3}, 1\right).$$

Normal output

A normal quantile transform applies the inverse Gaussian CDF to fitted cumulative probabilities.

Key intuition

Equal rank steps become equal probability steps, even when raw numerical gaps are unequal.

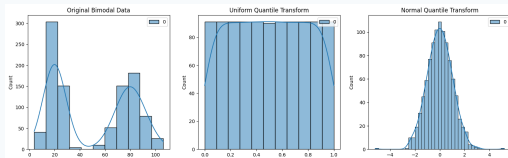
QuantileTransformer: Tool and Implementation

```
from sklearn.preprocessing import QuantileTransformer

np.random.seed(42)
d1 = np.random.normal(20, 5, 500)
d2 = np.random.normal(80, 10, 500)
bimodal = np.concatenate([d1, d2]).reshape(-1, 1)

qt_uniform = QuantileTransformer(
    output_distribution='uniform', n_quantiles=100
)
uniform_data = qt_uniform.fit_transform(bimodal)

qt_normal = QuantileTransformer(
    output_distribution='normal', n_quantiles=100
)
normal_data = qt_normal.fit_transform(bimodal)
```



Interpretation

The mapping preserves order but deliberately changes distances between observations.

Full-Data Practice: Quantile Transform on Sonar

Experiment

Apply uniform and normal quantile transforms to the same Sonar training split used in Lab 3.

Compare

- ▶ Distribution shape
- ▶ Rank preservation
- ▶ Outlier compression
- ▶ Validation performance

Leakage-safe requirement

Fit empirical quantiles on training data only. Test observations must be mapped with the fitted transformer.

Discussion

Which features benefit from a rank transform, and what information might be lost?

Other Transforms: Discretization and Polynomial Features

Discretization

Converts a continuous feature into bins, for example age 25 into the interval 20–30. It can help linear models represent threshold effects.

Polynomial features

Creates powers and interactions such as a^2 , b^2 , and ab , allowing linear models to express nonlinear relationships.

Modeling caution

Both methods expand modeling flexibility and can increase overfitting. Validate choices inside a pipeline.

KBinsDiscretizer: Three Strategies

Uniform

Equal-width intervals. For a 0–100 range and five bins: 0–20, 20–40, 40–60, 60–80, and 80–100.

Quantile

Approximately equal numbers of observations per bin. Useful for skewed data and stable sample sizes.

K-means

Uses one-dimensional clusters to place boundaries around high-density groups.

The bin label is a representation choice

Use `encode='ordinal'` for ordered bin indices or one-hot encoding when the model should not assume equal spacing between bins.

KBinsDiscretizer: Toy Income Dataset

Continuous values

Annual incomes in USD thousands:

$$x = (20, 25, 30, 60, 90).$$

We want three ordered categories that a simple linear model can use.

Uniform strategy

The training range is $90 - 20 = 70$.
Three equal-width bins each have width

$$70/3 \approx 23.33.$$

KBinsDiscretizer: Hand Calculation

Uniform boundaries

$[20, 43.33), [43.33, 66.67), [66.67, 90].$

Hence

$(20, 25, 30, 60, 90) \mapsto (0, 0, 0, 1, 2).$

What was lost?

Values 20, 25, and 30 become indistinguishable after binning. The model gains a threshold representation but loses within-bin variation.

KBinsDiscretizer: Tool and Implementation

Choose a strategy

- ▶ Uniform: fixed value intervals are meaningful
- ▶ Quantile: skewed data and balanced frequencies
- ▶ K-means: natural clusters are expected

How many bins?

Treat `n_bins` as a model hyperparameter and validate it; avoid bins with too few observations.

```
from sklearn.preprocessing import KBinsDiscretizer

continuous = np.random.randn(100, 1) * 10 + 50

disc_uniform = KBinsDiscretizer(
    n_bins=5, encode='ordinal', strategy='uniform'
)
disc_quantile = KBinsDiscretizer(
    n_bins=5, encode='ordinal', strategy='quantile'
)
disc_kmeans = KBinsDiscretizer(
    n_bins=5, encode='ordinal', strategy='kmeans',
    random_state=42
)
```

Full-Data Practice: Discretization on Sonar

Task

Choose several continuous Sonar features and compare uniform, quantile, and K-means bin boundaries.

Inspect

- ▶ Counts per bin
- ▶ Boundary stability across folds
- ▶ Information lost within bins

Evaluation question

Does discretization improve a linear classifier, and does it help or hurt a tree-based model?

PolynomialFeatures Transform

Definition

For two inputs (a, b) and degree 2, the transformed representation can include

$$a, b, a^2, ab, b^2.$$

When to use

Suspected curvature or interactions that should be captured by a linear model.

Key parameters

- ▶ degree
- ▶ interaction_only
- ▶ include_bias

Feature explosion

High degree creates many correlated features, increasing computation and overfitting. Degrees 2 or 3 are usually the practical limit.

PolynomialFeatures: Toy Dataset and Hand Calculation

Input and output

For rows $(2, 3)$ and $(4, 5)$ with degree 2:

$$(2, 3) \mapsto (2, 3, 4, 6, 9),$$

$$(4, 5) \mapsto (4, 5, 16, 20, 25).$$

Generated feature names

For inputs (a, b) and degree 2:

$$[a, b, a^2, ab, b^2].$$

The interaction ab allows the effect of one feature to depend on the other.

Scale before or after?

Create polynomial features first, then scale the expanded matrix before fitting a scale-sensitive model.

PolynomialFeatures: Tool and Implementation

Tool

`sklearn.preprocessing.
PolynomialFeatures`

Pipeline order

Generate polynomial terms, scale the expanded matrix, then fit the regularized estimator.

```
from sklearn.preprocessing import PolynomialFeatures

X_simple = np.array([[2, 3], [4, 5]])
poly = PolynomialFeatures(
    degree=2, include_bias=False
)
X_poly = poly.fit_transform(X_simple)

print(poly.get_feature_names_out())
print(X_poly)
```

Full-Data Practice: Polynomial Features on Sonar

Experiment

Build a pipeline with `PolynomialFeatures`, `StandardScaler`, and a regularized linear classifier.

Compare

Degree 1 versus degree 2; all terms versus interactions only.

Monitor complexity

Record the number of generated columns, training time, validation score, and the gap between training and validation performance.

Transforming the Target Variable (y)

Problem

Regression targets such as house prices or transaction counts are often right-skewed and heteroskedastic.

Solution

Train the model on a transformed target, such as $\log(1 + y)$, then inverse-transform predictions back to the original units.

Critical step

A prediction on the transformed scale is not yet a business prediction:

$$\log(\text{price}) \longrightarrow \exp(\widehat{\log(\text{price})}).$$

Safe tool

`TransformedTargetRegressor` automates transformation during fit and inversion during predict.

Target Transformation: Common Choices

Log transform	Box–Cox	Yeo–Johnson
<code>np.log1p</code> is effective for positive, right-skewed targets and handles zero because it calculates $\log(1 + y)$.	Estimates a power coefficient automatically, but requires strictly positive targets.	Supports negative, zero, and positive target values and is often the flexible default.

Choose the evaluation scale deliberately

Business errors are usually interpreted in original units, even when the model is trained on a transformed target.

Target Transform: Toy House-Price Dataset

Original target

House prices in USD thousands:

$$y = (100, 150, 400).$$

The large final value creates a right-skewed target and can dominate squared-error loss.

Chosen transformation

For a transparent calculation, use the natural logarithm:

$$y^* = \log(y).$$

Deployment requirement

Predictions must later be exponentiated to return to USD thousands.

Target Transform: Hand Calculation

Transform the targets

$$\log(100) = 4.605, \quad \log(150) = 5.011, \\ \log(400) = 5.991.$$

The raw gap 250 becomes approximately 0.98 on the log scale.

Invert a prediction

If the model predicts $\hat{y}^* = 5.30$, then

$$\hat{y} = \exp(5.30) \approx 200.3.$$

Interpretation

The final business prediction is approximately USD 200,300.

Target Transform: Tool

TransformedTargetRegressor

This meta-estimator couples a regression model with a target transformation and its inverse.

During fit

```
regressor.fit( $X$ ,  $f(y)$ ).
```

During predict

```
 $f^{-1}(\text{regressor.predict}(X))$ .
```

Why use the wrapper?

It prevents forgotten inverse transformations and keeps training and prediction behavior consistent.

TransformedTargetRegressor: Safe Implementation

```
from sklearn.compose import TransformedTargetRegressor
from sklearn.linear_model import LinearRegression

ttr = TransformedTargetRegressor(
    regressor=LinearRegression(),
    func=np.log1p,
    inverse_func=np.expm1
)

ttr.fit(X_train, y_train)
y_pred = ttr.predict(X_test)
# y_pred is already returned in original units
```

During fit

The regressor learns from $f(y)$.

During predict

The wrapper computes $f^{-1}(\widehat{f(y)})$ automatically.

Benefit

Transformation and inversion remain coupled to the estimator, reducing unit and deployment errors.

Full-Data Practice: California Housing Targets

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error

X, y = fetch_california_housing(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

ttr.fit(X_train, y_train)
y_pred = ttr.predict(X_test)
print(mean_absolute_error(y_test, y_pred))
```

Practice task

Compare LinearRegression with and without a log target transform using the same train/test split.

Report

Evaluate errors in original target units and inspect whether residual variance becomes more stable.

Summary and Key Takeaways

Represent inputs appropriately

- ▶ Scale numerical features
- ▶ Encode nominal and ordinal categories differently
- ▶ Transform strongly skewed distributions when justified

Control complexity and leakage

- ▶ Validate bins and polynomial degree
- ▶ Inverse-transform target predictions
- ▶ Fit every data-dependent transform on training data only

Default starting point

StandardScaler for ordinary numerical features, OneHotEncoder for nominal categories, and a leakage-safe pipeline for the complete workflow.

LECTURE 2

Q&A

Data Transform: questions and discussion